

Especificación del solver:

highOrderElectroActivationFoamImplicitJfNK

Un solver monodominio de alto orden, implícito, Jacobian-free Newton–Krylov en OpenFOAM con back-end de álgebra lineal Eigen

Pablo

7 de julio de 2026

Resumen

Este reporte especifica los algoritmos implementados en el solver de OpenFOAM `highOrderElectroActivationFoamImplicitJfNK`. El solver avanza las ecuaciones monodominio de la electrofisiología cardíaca acopladas al modelo celular iónico ten Tusscher–Noble–Noble–Panfilov (TNNP, 2004). El operador de difusión se discretiza con el marco de volúmenes finitos centrados en celda de OpenFOAM, opcionalmente mejorado a una reconstrucción de alto orden Local Radial-Extrapolation (LRE); la fuente iónica se evalúa ya sea en los centros de celda o en los puntos de cuadratura de celda de la LRE, con el ODE de reacción integrado en los puntos de cuadratura, una vez por celda y reconstruido, o por un híbrido consciente del frente (*front-aware*) que hace lo primero solo en el frente propagante y lo segundo en el resto (Sección 5); la integración temporal sigue un esquema θ (backward Euler o Crank–Nicolson); el sistema no lineal acoplado resultante puede resolverse por iteración de punto fijo de Picard, una linealización diagonal de I_{ion} , o un método Jacobian-Free Newton–Krylov (JFNK) con búsqueda de línea (*line search*) de retroceso de Armijo, valor inicial por extrapolación lineal y un *clamp* de bracket físico sobre la entrada al ODE iónico; el sistema lineal interno de JFNK se resuelve con un GMRES reiniciado propio con ortogonalización de Gram–Schmidt modificada, construido sobre la biblioteca de álgebra lineal dispersa Eigen; el ODE iónico por celda usa el integrador adaptativo RKF45 de OpenFOAM con tolerancias configurables; los tiempos de activación se registran por interpolación lineal a través del primer cruce de umbral y los puntos del benchmark se muestrean por ponderación de distancia inversa. Todos los parámetros algorítmicos se exponen a través de los diccionarios `constant/spatialIntegrationProperties` y `constant/timeIntegrationProperties`, y el driver Python compañero `run_convergence.py` genera estos diccionarios a partir de un único archivo de configuración. Este solver es algorítmicamente idéntico a su hermano basado en PETSc `highOrderElectroActivationFoamImplicitJfNK.PETSc`; la única diferencia es la biblioteca de álgebra lineal (Eigen vs. PETSc).

Índice

1. Notación	3
2. Ecuaciones de gobierno	3
2.1. PDE monodominio	3
2.2. ODE iónico	3
2.3. Ecuación MMS manufacturada usada para transferir la convergencia	4
2.4. Estímulo externo	4
3. Discretización temporal: el esquema θ	4
4. Discretización espacial	5
4.1. Laplaciano FV ortogonal estándar	5
4.2. Difusión LRE de alto orden	5
4.3. Matrices de masa	5

4.4. Reconstrucción de alto orden en los puntos de cuadratura iónicos	6
4.5. Promediado de alto orden de I_{ion} a celda	6
5. Integración de states en los puntos de cuadratura iónicos	6
5.1. Dos modos	6
5.2. Por qué el ODE de reacción debe resolverse en los puntos de Gauss del frente	6
5.3. Híbrido consciente del frente (front-aware)	7
5.4. Limitador de la reconstrucción de states	8
5.5. Claves de diccionario	8
6. Solver no lineal	9
6.1. Residuos compartidos y tests de parada	9
6.2. Método de punto fijo de Picard	9
6.3. Linealización diagonal de I_{ion}	10
6.4. Jacobian-Free Newton–Krylov	10
6.5. GMRES reiniciado propio	10
6.6. Búsqueda de línea de retroceso de Armijo	11
6.7. Valor inicial, clamp del ODE y rollback	11
7. Solver lineal: back-end Eigen	11
8. Integración del ODE iónico	11
9. Protección numérica del TNNP	12
10. Seguimiento de la activación y benchmarks	12
11. Diagrama de flujo del solver	13
12. Driver manufacturado: run_convergence.py	15
12.1. Geometría y barrido de parámetros	15
12.2. Método no lineal y controles de convergencia	15
12.3. Controles de JFNK y de diagonal-Ion	16
12.4. Solver lineal del PDE	17
12.5. Integrador del ODE de states	17
12.6. Parámetros LRE de alto orden por nivel-p	18
12.7. Diagnósticos y control de corrida	18
13. Resultados	18
13.1. Benchmark 2D estilo Niederer	19
13.2. Benchmark 3D estilo Niederer	20
A. Tabla de dimensiones	21
B. Mapa de archivos	22
C. Diferencias vs. el hermano PETSc	22

1. Notación

Símbolo	Significado	Unidades
$\Omega \subset \mathbb{R}^d$	Dominio espacial ($d \in \{2, 3\}$)	m
$\partial\Omega$	Frontera aislada (Neumann de flujo nulo)	—
$t \in (0, T]$	Tiempo	s
$V_m(\mathbf{x}, t)$	Voltaje transmembrana	V
$\mathbf{s}(\mathbf{x}, t) \in \mathbb{R}^{17}$	States iónicos del TNNP	mixtas
$I_{\text{ion}}(V_m, \mathbf{s})$	Densidad de corriente iónica total	V s^{-1}
$I_{\text{ext}}(\mathbf{x}, t)$	Densidad de corriente de estímulo externo	A m^{-3}
χ	Razón superficie-a-volumen de membrana	m^{-1}
C_m	Capacitancia de membrana por unidad de área	F m^{-2}
$\mathbf{D}(\mathbf{x})$	Tensor de conductividad de volumen	S m^{-1}
$\theta \in \{0.5, 1\}$	Parámetro del avance temporal (CN o BE)	—
Δt	Longitud del paso de tiempo externo	s
N_c	Número de celdas de malla	—
$\mathbf{V}^n \in \mathbb{R}^{N_c}$	Vector de V_m promediado en celda en t^n	V
$\mathbf{M}, \mathbf{K}, \mathbf{L}$	Matriz de masa, de rigidez, de difusión escalada	—
$\mathbf{A}, \mathbf{B}$	Operadores implícitos LHS / RHS del esquema θ	—
$\mathbf{F}(\mathbf{x})$	Residuo no lineal discreto a anular	—

2. Ecuaciones de gobierno

2.1. PDE monodominio

El voltaje de volumen se propaga por la ecuación estándar monodominio de reacción–difusión [1, 2]. En notación física de densidad de corriente puede escribirse

$$\chi C_m \frac{\partial V_m}{\partial t} = \nabla \cdot (\mathbf{D} \nabla V_m) - \chi C_m I_{\text{ion}}(V_m, \mathbf{s}) + I_{\text{ext}} \quad \text{en } \Omega \times (0, T], \quad (1)$$

sujeta a fronteras aisladas $\mathbf{n} \cdot (\mathbf{D} \nabla V_m) = 0$ en $\partial\Omega \times (0, T]$, y condición inicial $V_m(\mathbf{x}, 0) = -84 \text{ mV}$. El tensor de conductividad $\mathbf{D}$ es simétrico definido positivo y ya sea uniforme o leído como un `volTensorField` desde disco. El factor χC_m que multiplica a I_{ion} en (1) refleja la convención usada por la interfaz del modelo iónico: el solver almacena `Iion` como una tasa de voltaje con dimensiones V s^{-1} , no como una corriente volumétrica A m^{-3} .

Dividiendo (1) por χC_m se obtiene la forma realmente usada dentro del solver,

$$\frac{\partial V_m}{\partial t} = \mathcal{L} V_m + s(V_m, \mathbf{s}), \quad \mathcal{L} = \frac{1}{\chi C_m} \nabla \cdot (\mathbf{D} \nabla \cdot), \quad s = -I_{\text{ion}} + \frac{I_{\text{ext}}}{\chi C_m}. \quad (2)$$

Esto es exactamente lo que `rhsFromIion` ensambla en el solver.

2.2. ODE iónico

En cada punto de muestreo espacial el vector de states del TNNP $\mathbf{s} \in \mathbb{R}^{17}$ evoluciona según

$$\frac{d\mathbf{s}}{dt} = \mathbf{f}_{\text{TNNP}}(V_m, \mathbf{s}; \text{tissue}), \quad \mathbf{s}(0) = \mathbf{s}_0, \quad (3)$$

donde `tissue` selecciona `epicardialCells`, `mCells` o `endocardialCells`. Los 17 states rastreados son

$$\mathbf{s} = (V, K_i, Na_i, Ca_i, Xr_1, Xr_2, Xs, m, h, j, d, f, fCa, s, r, g, Ca_{SR}).$$

La corriente iónica total devuelta al PDE es la `Iion_cm` algebraica del TNNP [1].

2.3. Ecuación MMS manufacturada usada para transferir la convergencia

El driver de convergencia en `tutorials.electro/highOrderManufacturedFDAImplicitJfNK` resuelve un problema manufacturado con la misma estructura numérica que el solver de electro-activación: PDE implícito tipo monodominio, ODEs locales de states, fuente iónica no lineal, reconstrucción LRE opcional para V_m y I_{ion} , y acoplamiento no lineal seleccionable `Picard/diagonalIion/JfNK`. Los campos analíticos son

$$V_m^*(\mathbf{x}, t) = \sqrt{1+t} F(\mathbf{x}), \quad (4)$$

$$u_1^*(\mathbf{x}, t) = (1+t) G(\mathbf{x}) + V_m^*(\mathbf{x}, t), \quad (5)$$

$$u_2^*(\mathbf{x}, t) = \frac{1}{(1+t)\sqrt{G(\mathbf{x})}}, \quad (6)$$

$$u_3^*(\mathbf{x}, t) = 0, \quad (7)$$

con, por ejemplo, $F = \cos(\pi x) \cos(2\pi y)$ y $G = 1 + x y^2$ en 2D. En 3D, $F = \cos(\pi x) \cos(2\pi y) \cos(3\pi z)$ y $G = 1 + x y^2 z^3$. Las ecuaciones de states son

$$\dot{u}_1 = (u_1 + u_3 - V_m)^2 u_2^2 + \frac{1}{2} (u_1 + u_3 - V_m) u_2^2 (V_m - u_3), \quad (8)$$

$$\dot{u}_2 = -(u_1 + u_3 - V_m) u_2^3, \quad (9)$$

$$\dot{u}_3 = 0. \quad (10)$$

La fuente iónica manufacturada es

$$I_{\text{ion}}^{\text{MMS}}(V_m, \mathbf{u}) = -\frac{1}{2} (u_1 + u_3 - V_m) u_2^2 (V_m - u_3) + \frac{\beta}{\chi C_m} (V_m - u_3), \quad (11)$$

y la fuente del PDE usada por el solver manufacturado es $-I_{\text{ion}}^{\text{MMS}}$. El coeficiente β es el autovalor espacial del campo trigonométrico manufacturado bajo el tensor de conductividad constante:

$$\beta = \begin{cases} -\pi^2 D_{xx}, & d = 1, \\ -\pi^2 (D_{xx} + 4D_{yy}), & d = 2, \\ -\pi^2 (D_{xx} + 4D_{yy} + 9D_{zz}), & d = 3. \end{cases}$$

Así, el caso MMS no busca reproducir la fisiología del TNNP; es un problema controlado con solución exacta conocida que ejercita los mismos operadores discretos y caminos de solución no lineal usados por este solver.

2.4. Estímulo externo

La corriente externa es un escalón ventaneado en tiempo y en región:

$$I_{\text{ext}}(\mathbf{x}, t) = \begin{cases} I_0 & \mathbf{x} \in \Omega_{\text{stim}}^{(b)}, \quad t \in [t_0^{(b)}, t_0^{(b)} + \tau], \quad b = 1, \dots, B, \\ 0 & \text{en otro caso,} \end{cases} \quad (12)$$

con $I_0 = 50\,000 \text{ A m}^{-3}$ y $\tau = 2 \text{ ms}$ como valores por defecto del benchmark de Niederer.

3. Discretización temporal: el esquema θ

El tiempo se avanza de t^n a $t^{n+1} = t^n + \Delta t$ por

$$\frac{\mathbf{V}^{n+1} - \mathbf{V}^n}{\Delta t} = \theta \mathbf{L} \mathbf{V}^{n+1} + (1-\theta) \mathbf{L} \mathbf{V}^n + \theta \mathbf{s}^{n+1} + (1-\theta) \mathbf{s}^n. \quad (13)$$

<code>implicitScheme</code>	θ	Esquema
<code>backwardEuler</code>	1	Primer orden, L-estable
<code>crankNicolson</code>	1/2	Segundo orden, A-estable

Tras multiplicar por Δt^{-1} la implementación forma

$$[\mathbf{M}/\Delta t - \theta \mathbf{L}] \mathbf{V}^{n+1} = [\mathbf{M}/\Delta t + (1 - \theta) \mathbf{L}] \mathbf{V}^n + \theta \mathbf{s}^{n+1} + (1 - \theta) \mathbf{s}^n, \quad (14)$$

con los dos operadores `Eigen SparseMatrix<scalar, RowMajor>`

$$\mathbf{A} = \mathbf{M}/\Delta t - \theta \mathbf{L}, \quad \mathbf{B} = \mathbf{M}/\Delta t + (1 - \theta) \mathbf{L}. \quad (15)$$

$\mathbf{A}$ y $\mathbf{B}$ se reconstruyen en cada paso de tiempo copiando $\mathbf{M}$, escalando por Δt^{-1} y sumando $\pm \theta \mathbf{L}$. Cuando $\theta = 1$ (BE) la mitad explícita de la difusión se anula y $\mathbf{B} = \mathbf{M}/\Delta t$. La difusión escalada es $\mathbf{L} = \mathbf{K}/(\chi C_m)$.

4. Discretización espacial

4.1. Laplaciano FV ortogonal estándar

`assembleStandardOrthogonalStiffnessMatrix` computa el Laplaciano clásico de 5/7 puntos. Por cada cara interna f con área de cara $\mathbf{S}_f$,

$$a_f = \frac{|\mathbf{S}_f| (\mathbf{n}_f \cdot \mathbf{D}_f \cdot \hat{\mathbf{d}}_f)}{|\mathbf{d}_f|}, \quad \mathbf{D}_f = \frac{1}{2} (\mathbf{D}_{\text{own}} + \mathbf{D}_{\text{nei}}), \quad (16)$$

aportando $-a_f/V_{\text{own}}$ a $\mathbf{K}[\text{own}, \text{own}]$, $+a_f/V_{\text{own}}$ a $\mathbf{K}[\text{own}, \text{nei}]$, y el par transpuesto a la fila del vecino. Los patches `zeroGradient/empty` aportan flujo nulo. Los patches `fixedValue` aportan solo a la diagonal del dueño (owner).

4.2. Difusión LRE de alto orden

Cuando `useHighOrder_Vm = true` y $d > 1$, `assembleHighOrderStiffnessMatrix` integra el flujo en una cuadratura de cara con gradientes reconstruidos:

$$\Phi_f = \sum_{q \in \mathcal{Q}_f} w_q |\mathbf{S}_f| \mathbf{n}_f \cdot \mathbf{D}_f \cdot \left(\sum_{j \in \mathcal{S}_f} \mathbf{g}_{f,q,j} \mathbf{V}_j \right), \quad (17)$$

donde $\mathcal{S}_f$ es el stencil de cara de la LRE (~ 15 – 50 celdas) y $\mathbf{g}_{f,q,j} = \text{QRGradFaceGPCoeffs}$. La contribución de cada $\mathbf{V}_j$ se suma a la fila `own` con peso $+\Phi_{f,j}/V_{\text{own}}$ y a la fila `nei` con peso $-\Phi_{f,j}/V_{\text{nei}}$, dando un Laplaciano semi-definido negativo.

4.3. Matrices de masa

Dos opciones:

Lumped (`massMatrix lumped`). `assembleDiagonalMassMatrix` escribe $\mathbf{M} = \mathbf{I}$.

Consistente HO (`massMatrix consistent`). `assembleConsistentMassMatrixHO` llena

$$\mathbf{M}[i, j] = \sum_{q \in \mathcal{Q}_i} \frac{w_q}{\sum_{q'} w_{q'}} \phi_j(\mathbf{x}_q), \quad \phi_j(\mathbf{x}_q) = \delta_{ij} + \mathbf{g}_{i,j} \cdot \mathbf{d}_q + \frac{1}{2} \mathbf{H}_{i,j} : (\mathbf{d}_q \otimes \mathbf{d}_q) + \frac{1}{6} \mathbf{T}_{i,j}^{(3)} : (\mathbf{d}_q \otimes \mathbf{d}_q \otimes \mathbf{d}_q), \quad (18)$$

con $\mathbf{d}_q = \mathbf{x}_q - \mathbf{C}_i$. Una salvaguarda degrada silenciosamente a la forma `lumped` cuando `useHighOrder_Vm = false` o $d \leq 1$.

4.4. Reconstrucción de alto orden en los puntos de cuadratura iónicos

Si `useHighOrder.Iion = true`, el ODE iónico se integra en los puntos de cuadratura de celda. V_m se reconstruye ahí por la misma expansión de Taylor que en (18):

$$V_m(\mathbf{x}_q) \approx V_{m,c} + (\nabla V_m)_c \cdot \mathbf{d}_q + \frac{1}{2} \mathbf{H}_c : (\mathbf{d}_q \otimes \mathbf{d}_q) + \frac{1}{6} \mathbf{T}_c^{(3)} (\mathbf{d}_q \otimes \mathbf{d}_q \otimes \mathbf{d}_q), \quad (19)$$

implementado en `reconstructVmAtIionIntegrationPoints`.

4.5. Promediado de alto orden de I_{ion} a celda

`averageIionIntegrationPointsToCells` contrae el I_{ion} por punto de cuadratura de vuelta a celdas:

$$I_{\text{ion},i} = \frac{\sum_{q \in \mathcal{Q}_i} w_q I_{\text{ion},q}}{\sum_{q \in \mathcal{Q}_i} w_q}. \quad (20)$$

5. Integración de states en los puntos de cuadratura iónicos

La cuadratura de alto orden (19) evalúa $I_{\text{ion}}(V_m, \mathbf{s})$ en los puntos de cuadratura de celda $\mathbf{x}_q$, así que también necesita ahí el vector de states iónicos $\mathbf{s}$. *Cómo* llegan los states a los puntos de cuadratura lo gobierna `stateIntegrationMode` y, ortogonalmente, `frontAwareHybrid`. Esta elección resulta decisiva para un frente cardíaco propagante, y esta sección documenta tanto el algoritmo como la razón por la que importa.

5.1. Dos modos

Sea $\mathcal{Q}_i = \{\mathbf{x}_q\}$ el conjunto de cuadratura de la celda Ω_i con pesos w_q , y sea $\mathbf{s}_c$ un state centrado en celda.

gaussPointODE (heredado). El ODE rígido (3) se integra *independientemente en cada punto de cuadratura*, manejado por el potencial reconstruido localmente:

$$\mathbf{s}_q^{n+1} = \text{ODEsolve}(\mathbf{s}_q^n, V_m(\mathbf{x}_q); \Delta t), \quad I_{\text{ion},q} = \phi(V_m(\mathbf{x}_q), \mathbf{s}_q^{n+1}). \quad (21)$$

Costo: N_q integraciones rígidas por celda.

cellCentredReconstruct. El ODE se integra *una vez por celda* en el centro de celda, y los states centrados en celda resultantes se reconstruyen a los puntos de cuadratura por una LRE dedicada (`LREInterp_states`, orden fijado por `highOrderCoeffs/LRECoeffs_states`):

$$\mathbf{s}_c^{n+1} = \text{ODEsolve}(\mathbf{s}_c^n, \bar{V}_{m,c}; \Delta t), \quad \hat{\mathbf{s}}(\mathbf{x}_q) = \mathcal{R}^{(p)}[\{\mathbf{s}_c^{n+1}\}](\mathbf{x}_q), \quad I_{\text{ion},q} = \phi(V_m(\mathbf{x}_q), \hat{\mathbf{s}}(\mathbf{x}_q)). \quad (22)$$

Costo: una integración rígida por celda — un factor N_q más barato, lo cual para las costosas integraciones TNNP es la motivación. Los dos modos difieren solo en el argumento del mapa no lineal $V_m \mapsto \mathbf{s}$: puntual $V_m(\mathbf{x}_q)$ en (21), versus el promedio de celda $\bar{V}_{m,c}$ en (22).

5.2. Por qué el ODE de reacción debe resolverse en los puntos de Gauss del frente

Para un campo de reacción suave los dos modos coinciden hasta el orden de reconstrucción, y `cellCentredReconstruct` es preferible por su velocidad. En un *frente cardíaco propagante* no coinciden, y solo la evaluación puntual (21) es admisible. La razón es la fuerte no linealidad de la activación de sodio.

El upstroke (fase de despolarización rápida) lo lleva $I_{\text{Na}} = g_{\text{Na}} m^3 h j (V_m - E_{\text{Na}})$, cuya compuerta de activación relaja hacia el sigmoide empujado

$$m_\infty(V_m) = \left[1 + \exp((V_{1/2} - V_m)/k) \right]^{-1}, \quad V_{1/2} \approx -40 \text{ mV}, \quad k \approx 6 \text{ mV}. \quad (23)$$

Por debajo de su inflexión ($V_m < V_{1/2}$) — exactamente el “pie” sub-umbral donde se deciden la ignición y la conducción — m_∞ es *convexa*. Escribiendo $\langle \cdot \rangle_i = (\sum_q w_q \cdot) / \sum_q w_q$ para el promedio de cuadratura de celda, de modo que $\bar{V}_{m,i} = \langle V_m \rangle_i$, los dos modos evalúan la compuerta como

$$\underbrace{\langle m_\infty(V_m) \rangle_i}_{\text{gaussPointODE}} \geq \underbrace{m_\infty(\langle V_m \rangle_i)}_{\text{cellCentredReconstruct}}, \quad (24)$$

por la desigualdad de Jensen para la m_∞ convexa. Como $I_{\text{Na}} \propto m^3$, la brecha se amplifica al cubo: el camino de promedio de celda *sub-estima sistemáticamente* la corriente entrante regenerativa justo donde V_m varía dentro de una celda, es decir en el frente de onda y el borde del estímulo. Reconstruir los states después del ODE (como en (22)) no puede repararlo — el sesgo se comete *dentro* del ODE, cuya entrada ya era el potencial promediado.

Una estimación cuantitativa. Sea $\Delta V = \max_q V_m(\mathbf{x}_q) - \min_q V_m(\mathbf{x}_q)$ el rango sub-celda de V_m a través de una celda que abarca el pie del upstroke, y tratemos a V_m como aproximadamente uniforme sobre la celda (varianza $\sigma^2 \simeq \Delta V^2/12$). Una expansión de punto medio del promedio da

$$\langle m_\infty \rangle - m_\infty(\bar{V}) \simeq \frac{1}{2} m_\infty''(\bar{V}) \sigma^2 = m_\infty''(\bar{V}) \frac{\Delta V^2}{24}. \quad (25)$$

En el pie exponencial $m_\infty \approx e^{(V_m - V_{1/2})/k}$, de modo que $m_\infty''/m_\infty = 1/k^2$ y, usando $I_{\text{Na}} \propto m^3$,

$$\boxed{\frac{\delta I_{\text{Na}}}{I_{\text{Na}}} \approx 3 \cdot \frac{\Delta V^2}{24 k^2} = \frac{\Delta V^2}{8 k^2}}. \quad (26)$$

Con $k = 6 \text{ mV}$ y un rango sub-celda $\Delta V \sim 30 \text{ mV}$ a través de un frente sub-resuelto, (26) da $\delta I_{\text{Na}}/I_{\text{Na}} \approx 900/288 \approx 3$ — la estimación linealizada ya es $O(1)$, es decir la evaluación de promedio de celda sub-dispara la corriente de sodio por un factor de orden unidad o más. Siguen dos modos de falla, ambos observados y documentados en la nota compañera `stateIntegrationMode_conductionDiagnosis.tex`:

1. **Falla de ignición:** un estímulo nominal queda efectivamente sub-umbral; V_m sube pero el upstroke regenerativo nunca dispara.
2. **Bloqueo de conducción:** una vez encendido, cada celda del frente aporta muy poca carga entrante para llevar a sus vecinas aguas abajo al umbral — un desajuste source-sink (fuente-sumidero) inducido numéricamente que estanca la onda.

Refinar Δt hace el frente más filoso (mayor ΔV) y por lo tanto *agrava* el déficit, razón por la cual el camino `cellCentredReconstruct` puro falla a Δt chico mientras que un sobre-disparo de Δt grueso puede enmascararlo. La conclusión es inevitable: en alto orden sobre un frente filoso el ODE de reacción *debe* resolverse en los puntos de Gauss de las celdas del frente; solo los volúmenes restantes (suaves) pueden reconstruirse de forma barata.

5.3. Híbrido consciente del frente (front-aware)

`frontAwareHybrid` (por defecto `false`, ortogonal a `stateIntegrationMode`) realiza exactamente esa conclusión: resuelve el ODE en los puntos de Gauss en la banda delgada de celdas de *frente* y en el centro de celda (con reconstrucción) en el resto. Una celda i es de *frente* cuando su rango sub-celda de V_m excede `hybridFrontVmRange` o está estimulada,

$$\text{frente}_i \iff \left(\max_q V_m(\mathbf{x}_q) - \min_q V_m(\mathbf{x}_q) \right) > \text{hybridFrontVmRange} \vee (I_{\text{ext}})_i \neq 0. \quad (27)$$

Los states persistentes se llevan en los puntos de Gauss (como en `gaussPointODE`); la reconstrucción de celdas suaves deriva sus valores de celda promediando por cuadratura ese portador. Por evaluación no lineal la rama es:

- (a) celdas de frente: un ODE por punto de Gauss, V_m local (como (21));
- (b) celdas suaves: un ODE en el centro de celda desde el state promedio de celda;
- (c) reconstruir los states de las celdas suaves a sus puntos de Gauss, dejando intactos los states de Gauss del frente;
- (d) evaluar I_{ion} en todos los puntos de Gauss y promediar a la celda.

Congelar por paso de tiempo (evitando un ciclo límite). La clasificación de frente (27) es un *switch discreto dependiente de la solución*. Si se recalcula dentro del bucle no lineal, las celdas cerca del umbral alternan frente↔suave entre iteraciones; como las dos ramas computan I_{ion} por caminos distintos, el término fuente salta discretamente y el mapa de punto fijo de Picard se vuelve discontinuo, produciendo un *ciclo límite de período 2* no convergente (observado como $\text{iter}_{k+2} \equiv \text{iter}_k$ en `nonlinearResiduals.dat`). La clasificación se computa por lo tanto *una vez por paso de tiempo* desde el V_m^n committeado y se *congela* para todo el solve no lineal, y la banda se dilata `hybridFrontDilation` capas de celdas (vía `mesh.cellCells()`) para que el frente permanezca dentro de la zona resuelta en Gauss a medida que avanza. Congelar es seguro porque el frente se mueve mucho menos que una celda por paso (para el benchmark 2D, $\sim 0,09$ mm/ms, es decir $\sim 0,009$ celda por paso a $\Delta t = 10^{-4}$ s, $\Delta x = 0,5$ mm).

5.4. Limitador de la reconstrucción de states

La reconstrucción de alto orden ($p3$) de los states rígidos del TNNP a través de un frente sub-resuelto sobre-dispara (Gibbs), y un state llevado muy fuera de su rango físico puede desbordar (overflow) el álgebra iónica (observado como un SIGFPE de PETSc). `stateReconstructLimiter` (por defecto `true`, activo para `cellCentredReconstruct` y las celdas suaves del híbrido, independiente de `frontAwareHybrid`) aplica una cota tipo Barth–Jespersen: cada valor reconstruido se recorta al rango de la celda y sus vecinas de cara,

$$\hat{s}(\mathbf{x}_q) \leftarrow \text{clip}\left(\hat{s}(\mathbf{x}_q), \min_{c' \in \{i\} \cup \mathcal{N}(i)} s_{c'}, \max_{c' \in \{i\} \cup \mathcal{N}(i)} s_{c'}\right). \quad (28)$$

No se crea ningún extremo nuevo, así que los states reconstruidos permanecen dentro del rango físicamente realizado (evolucionado por el ODE) y el álgebra iónica nunca ve una entrada patológica; en regiones suaves la reconstrucción ya cae dentro del rango y el limitador queda inactivo, preservando el orden ahí.

5.5. Claves de diccionario

Clave (en <code>implicitHOCoeffs</code>)	Significado	Por defecto
<code>stateIntegrationMode</code>	<code>gaussPointODE</code> / <code>cellCentredReconstruct</code>	<code>cellCentredReconstruct</code>
<code>frontAwareHybrid</code>	ODEs en Gauss en el frente, reconstrucción en el resto	<code>false</code>
<code>hybridFrontVmRange</code>	rango sub-celda de V_m que marca celda de frente [V]	5×10^{-3}
<code>hybridFrontDilation</code>	capas de celda agregadas alrededor de la banda de frente	2
<code>stateReconstructLimiter</code>	limitador Barth–Jespersen en la reconstrucción	<code>true</code>

`LRECoeffs_states` (en `highOrderCoeffs`) fija el orden de reconstrucción; es requerido por `cellCentredReconstruct` y por el híbrido.

6. Solver no lineal

En cada paso de tiempo externo la incógnita es el nuevo vector de voltaje $\mathbf{x} = \mathbf{V}^{n+1}$. Para cualquier candidato $\mathbf{x}$ el solver primero reconstruye el campo V_m correspondiente, luego llama a `evaluateNonlinearFields`. Esa rutina resetea los states iónicos a sus valores al inicio del paso $\mathbf{s}^n$, integra los ODEs locales de t^n a t^{n+1} usando el voltaje candidato, obtiene $I_{\text{ion}}(\mathbf{x})$, y opcionalmente promedia los valores de puntos de cuadratura de vuelta a celdas. Esto da el residuo no lineal

$$\mathbf{F}(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{r}(I_{\text{ion}}(\mathbf{x})), \quad \mathbf{x} = \mathbf{V}^{n+1}, \quad (29)$$

con

$$\mathbf{r}(I_{\text{ion}}) = \mathbf{B}\mathbf{V}^n + \theta \left[-I_{\text{ion}}(\mathbf{x}) + I_{\text{ext}}^{n+1}/(\chi C_m) \right] + (1 - \theta) \left[-I_{\text{ion}}^n + I_{\text{ext}}^n/(\chi C_m) \right]. \quad (30)$$

Los tres métodos no lineales difieren solo en cómo llevan a $\mathbf{F}(\mathbf{x})$ hacia cero.

6.1. Residuos compartidos y tests de parada

Cada método registra los mismos diagnósticos:

$$R_{\text{cpl}} = \frac{\|\mathbf{F}(\mathbf{x}_k)\|_2}{\|\mathbf{x}_k\|_2 + \epsilon}, \quad (31)$$

$$R_{V_m} = \frac{\|\mathbf{V}_k - \mathbf{V}_{k-1}\|_2}{\|\mathbf{V}_{k-1}\|_2 + \epsilon}, \quad (32)$$

$$R_{I_{\text{ion}}} = \frac{\|I_{\text{ion},k} - I_{\text{ion},k-1}\|_2}{\|I_{\text{ion},k-1}\|_2 + \epsilon}, \quad (33)$$

$$R_s = \max_j \frac{\|\mathbf{s}_{j,k} - \mathbf{s}_{j,k-1}\|_2}{\|\mathbf{s}_{j,k-1}\|_2 + \epsilon}. \quad (34)$$

La implementación real usa `relativeL2Difference` para R_{V_m} y $R_{I_{\text{ion}}}$, y `maxStateRelativeL2Difference` para el máximo de states. Una iteración no lineal solo puede parar tras `minNonlinearIterations`; la convergencia luego requiere `nonlinearTolerance`, `nonlinearVmTolerance` y `nonlinearIionTolerance`. El criterio de states `nonlinearStatesTolerance` se incluye solo cuando `nonlinearRequireStatesConvergence=true`.

6.2. Método de punto fijo de Picard

La rama Picard trata la fuente como retrasada (lagged) durante el solve del PDE. En la iteración k los states y la fuente iónica se computan desde $\mathbf{x}_k$, luego el PDE lineal

$$\mathbf{A}\mathbf{V}_* = \mathbf{r}(I_{\text{ion}}(\mathbf{x}_k)) \quad (35)$$

se resuelve con el solver disperso Eigen seleccionado por `linearSolver`. El siguiente candidato se relaja,

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \omega(\mathbf{V}_* - \mathbf{x}_k), \quad \omega = \text{nonlinearRelaxation}. \quad (36)$$

Tras la actualización el solver re-evalúa states y I_{ion} en $\mathbf{x}_{k+1}$ y chequea los residuos de arriba. En este solver de electro-activación, a diferencia de un estudio Picard MMS puro, un solve Picard fallido hace rollback (reversión) a menos que `nonlinearAcceptUnconverged=true`.

Ventajas. Picard es simple, barato por iteración no lineal, y usa el mismo solver lineal del PDE que la formulación implícita clásica. Es una línea base útil y a menudo es adecuado cuando Δt es chico o la fuente iónica es débilmente no lineal en un paso de tiempo.

Desventajas. La derivada de la fuente respecto a V_m se ignora en la matriz del PDE, así que la convergencia puede ser lenta cerca del upstroke del potencial de acción o para Δt grande. Un residuo chico puede requerir muchas iteraciones, y puede necesitarse sub-relajación para robustez.

6.3. Linealización diagonal de I_{ion}

La rama `diagonalIion` es Picard más una aproximación local por diferencias finitas de la derivada de la fuente. Definir

$$S(V_m) = -I_{\text{ion}}(V_m, \mathbf{s}(V_m)) + I_{\text{ext}}/(\chi C_m).$$

El solver aproxima, celda a celda,

$$S'_i(V_{m,k}) \approx -\frac{I_{\text{ion},i}(V_{m,k} + \varepsilon_d) - I_{\text{ion},i}(V_{m,k})}{\varepsilon_d}, \quad \varepsilon_d = \text{diagonalIionEpsilon}. \quad (37)$$

El residuo linealizado $\mathbf{A}\mathbf{x} - \theta S(\mathbf{x})$ da el sistema

$$(\mathbf{A} - \theta \text{diag}(S'_k)) \mathbf{x}_{k+1} = \mathbf{r}(I_{\text{ion}}(\mathbf{x}_k)) - \theta \text{diag}(S'_k) \mathbf{x}_k. \quad (38)$$

La actualización aún puede relajarse con `nonlinearRelaxation`, y se usa la misma lógica de convergencia y rollback que en Picard.

Ventajas. Captura la rigidez local dominante de $I_{\text{ion}}(V_m)$ sin construir un Jacobiano completo. A menudo reduce el número de iteraciones no lineales respecto a Picard preservando una matriz escalar dispersa del PDE.

Desventajas. Solo se incluye la sensibilidad diagonal local de celda. Los acoplamientos a través de los states del ODE, el promediado de cuadratura de alto orden y la reconstrucción espacial no se representan en la matriz. El paso de diferencias finitas `diagonalIionEpsilon` debe elegirse por encima de los niveles de ODE/ruido pero suficientemente chico para ser una derivada útil.

6.4. Jacobian-Free Newton–Krylov

JFNK aplica Newton a (29):

$$\mathbf{J}(\mathbf{x}_k) \boldsymbol{\delta}_k = -\mathbf{F}(\mathbf{x}_k), \quad \mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \boldsymbol{\delta}_k,$$

pero nunca ensambla $\mathbf{J}$. GMRES ve solo productos matriz-vector aproximados por

$$\mathbf{J}(\mathbf{x}_k) \mathbf{v} \approx \frac{\mathbf{F}(\mathbf{x}_k + \varepsilon \mathbf{v}) - \mathbf{F}(\mathbf{x}_k)}{\varepsilon}, \quad \varepsilon = \varepsilon_0 \frac{1 + \|\mathbf{x}_k\|_2}{\|\mathbf{v}\|_2}, \quad (39)$$

con $\varepsilon_0 = \text{jfnkEpsilon}$. Cada evaluación de $\mathbf{F}$ vuelve a correr los ODEs y recomputa I_{ion} desde el mismo snapshot de state al inicio del paso. Esto es costoso pero da un residuo no lineal genuinamente acoplado.

6.5. GMRES reiniciado propio

`solveGMRES` implementa GMRES(m) con Gram–Schmidt modificado. Resuelve el problema de mínimos cuadrados de Krylov para la corrección de Newton usando a lo sumo `jfnkMaxKrylovIterations` vectores base por reinicio y a lo sumo `jfnkMaxRestarts` reinicios. El test de parada lo controla `jfnkLinearTolerance`. El método es “matrix-free” (sin matriz) aquí porque el operador lineal es la envoltura (shell) de diferencias finitas de (39), no una matriz dispersa almacenada.

Ventajas. JFNK ve la dependencia no lineal completa $V_m \mapsto \mathbf{s}(V_m) \mapsto I_{\text{ion}}(V_m, \mathbf{s})$ y usualmente tiene una convergencia no lineal mucho mejor que Picard en regímenes rígidos. También evita formar explícitamente un Jacobiano denso o complicado.

Desventajas. Cada vector de Krylov requiere una evaluación adicional del residuo, y por ende una nueva integración de ODE en cada celda o punto de cuadratura. El método es sensible a la perturbación de diferencias finitas, a las tolerancias del ODE y a las elecciones de la búsqueda de línea. En esta implementación Eigen el GMRES interno de JFNK no tiene preconditionador.

6.6. Búsqueda de línea de retroceso de Armijo

La actualización cruda de Newton puede aumentar el residuo no lineal cuando el modelo lineal es pobre. Si `jfnkLineSearch=true`, el solver prueba

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha \boldsymbol{\delta}_k, \quad \alpha = \text{nonlinearRelaxation}, \alpha/2, \alpha/4, \dots$$

hasta que se cumple la condición de descenso suficiente de Armijo:

$$\|\mathbf{F}(\mathbf{x}_k + \alpha \boldsymbol{\delta}_k)\|_2^2 \leq (1 - 2c\alpha) \|\mathbf{F}(\mathbf{x}_k)\|_2^2, \quad c = \text{jfnkArmijoC}. \quad (40)$$

El retroceso para tras `jfnkLineSearchMaxIter` intentos o cuando la longitud de paso alcanza `jfnkLineSearchAlphaMin`. Esto globaliza Newton: los pasos grandes de Newton se aceptan cuando reducen el residuo, en caso contrario se amortiguan.

6.7. Valor inicial, clamp del ODE y rollback

El primer valor inicial para el paso de tiempo es ya sea $\mathbf{V}^n$ o la extrapolación lineal

$$\mathbf{x}_0 = \mathbf{V}^n + \frac{\Delta t}{\Delta t_{\text{prev}}} (\mathbf{V}^n - \mathbf{V}^{n-1}), \quad (41)$$

seleccionado por `jfnkInitGuessOrder`. Se recorta a `[jfnkInitGuessVmMin, jfnkInitGuessVmMax]`. El mismo bracket también puede aplicarse a los valores de voltaje pasados al solver del ODE mediante `jfnkClampODEInput`; esto protege al modelo iónico durante las sondas de diferencias finitas de JFNK.

Si el bucle no lineal alcanza su presupuesto de iteraciones sin satisfacer los tests de convergencia, `nonlinearAcceptUnconverged` decide si el último iterado se committea. Con el valor por defecto `false`, el solver hace rollback de V_m , I_{ion} y todos los states iónicos al inicio del paso de tiempo. El rollback evita que un solve no lineal malo contamine el resto de la simulación de activación.

7. Solver lineal: back-end Eigen

El envoltorio `solveSparseSystem` maneja cualquiera de dos solvers dispersos de Eigen sobre el operador explícito $\mathbf{A}$ (camino Picard / diagonal):

Clave <code>linearSolver</code>	Tipo Eigen	Tipo
<code>SparseLU</code>	<code>Eigen::SparseLU</code>	LU directo
<code>BiCGSTAB</code>	<code>Eigen::BiCGSTAB + IncompleteLUT</code>	Iterativo

Ambos comparten las claves de diccionario `tolerance` y `maxIterations` (la primera la ignora `SparseLU` por ser directo). La rama matrix-free de JFNK nunca instancia estos envoltorios; en cambio llama al GMRES propio (Sección 5.2) sobre la envoltura del Jacobiano FD.

8. Integración del ODE iónico

Para cada celda o punto de cuadratura de celda, el ODE del TNNP (3) se avanza con el integrador adaptativo de OpenFOAM seleccionado vía la clave `solver`. Por defecto RKF45 [6]. Claves leídas de `constant/timeIntegrationProperties`:

Clave	Significado	Por defecto
<code>solver</code>	Integrador ODE concreto	RKF45
<code>absTol</code>	Tolerancia absoluta por state	10^{-9}
<code>relTol</code>	Tolerancia relativa por state	10^{-6}
<code>maxSteps</code>	Tope de sub-pasos por llamada externa	10^4
<code>initialODEStep</code>	Sub-paso inicial (ms)	10^{-5}

`relTol` maneja el piso de ruido del Jacobiano matrix-free: $\eta_F \sim \text{relTol}$. Ajustar `relTol` más fino compra una tolerancia externa alcanzable más baja a costa de más sub-pasos de RKF45 por iteración de Newton.

9. Protección numérica del TNNP

El switch maestro `TNNPNumericalProtection` (por defecto `false`) condiciona una guarda local invocada en cada entrada externa al kernel TNNP embebido (`calculateCurrent`, `solveODE_start`, `solveODE_end`, `derivatives`).

Clave	Tipo	Por defecto
<code>TNNPNumericalProtection</code>	Switch	<code>false</code>
<code>TNNPCLampGates</code>	Switch	<code>true</code>
<code>TNNPCLampVmForRates</code>	Switch	<code>true</code>
<code>TNNPLogNumericalProtection</code>	Switch	<code>true</code>
<code>TNNPConcentrationFloor</code>	scalar (mM)	10^{-12}
<code>TNNPVMInForRates</code>	scalar (mV)	-200
<code>TNNPVMaxForRates</code>	scalar (mV)	$+100$
<code>TNNPVoltageZeroEps</code>	scalar (mV)	10^{-8}
<code>TNNPMaxProtectionLogEntries</code>	label	10^6
<code>TNNPProtectionLog</code>	word	<code>TNNP_numericalProtection_summary.dat</code>

Cuando la protección está habilitada, las guardas aplicadas en orden son:

1. **V no finito:** `!std::isfinite(V)` $\rightarrow$ resetear a $-86,2$ mV.
2. **Clamp de V para las tasas** (si `TNNPCLampVmForRates`): recortar a `[TNNPVMInForRates, TNNPVMaxForRates]`.
3. **Evitación de singularidad en $V = 0$:** empujar V fuera de cero por `TNNPVoltageZeroEps` (preservando el signo) cuando $|V|$ cae por debajo de ese umbral.
4. **Piso de concentración** en K_i, Na_i, Ca_i, Ca_{SR} : entradas no finitas o por debajo del piso se resetean a `TNNPConcentrationFloor`.
5. **Clamp de compuertas** en $Xr_1, Xr_2, Xs, m, h, j, d, f, fCa, s, r, g$ (si `TNNPCLampGates`): no finitas a cero, recortar a `[0, 1]`.

La implementación embebida almacena un conteo de correcciones acotado y emite un archivo de resumen compacto al salir; no depende de utilidades externas de logging del modelo iónico.

10. Seguimiento de la activación y benchmarks

Tiempo de activación por celda. El primer cruce de V_{thr} en un paso de tiempo se registra por interpolación lineal:

$$w = \text{clip}\left(\frac{V_{\text{thr}} - V^n}{V^{n+1} - V^n}, 0, 1\right), \quad t_{\text{act}} = t^n + w \Delta t, \quad (42)$$

con una bandera por celda que se limpia tras el primer cruce para ignorar cruces posteriores.

Terminación temprana en P8. Cuando `stopAfterPointActivation = true`, se monitorea la celda más cercana a `stopActivationPoint` (P8 en el benchmark de Niederer) y `effectiveEndTime` se fija a $t_{\text{act}} + \text{stopDelayAfterActivation}$ una vez que su $t_{\text{act}} > 0$.

Muestreo de la diagonal. `sampleIDW` realiza interpolación ponderada por distancia inversa en `nDiagonalSamples` puntos entre `diagonalStart` y `diagonalEnd`, con $k = \text{sampleNeighbours}$, $p = \text{samplePower}$. Como una muestra de la diagonal promedia sus k celdas más cercanas y las celdas no activadas contribuyen $t_{\text{act}} = 0$, una muestra observada *en vivo* sube desde abajo a medida que el frente llena su vecindario; el valor final (las k vecinas activadas) es el correcto.

Salida de activación en vivo. `writeActivationSamples` se llama una vez antes del bucle de tiempo (para que `diagonalActivationTime.dat` y `P8_activationTime.dat` existan desde $t = 0$ con todo punto pendiente) y de nuevo cada vez que el conteo de celdas activadas aumenta, sobrescribiendo los archivos. La corrida puede así monitorearse a medida que el frente avanza; el driver expone el archivo en vivo en cada directorio de corrida mediante un enlace simbólico que se reemplaza por una copia real al final.

Detección de falla de activación. Cuando `abortOnActivationFailure = true` el solver para temprano (limpiamente, exit 0) si la activación falla, evitando una corrida completa hasta `endTime`. Dos criterios, usando el conteo corriente de celdas activadas N_a y el tiempo t_* de la última activación nueva:

- **sin ignición:** $N_a = 0$ y $t > \text{activationIgnitionTimeout}$;
- **bloqueo de conducción:** $N_a > 0$, el punto de parada aún no se alcanzó, y $t - t_* > \text{activationStallTime}$.

El resultado se registra siempre en `postProcessing/.../activationStatus.dat` como uno de OK, FAILED_NO_IGNITION, FAILED_CONDUCTION_BLOCK o INCOMPLETE (con el tiempo final y N_a). El driver lee este archivo y, ante una falla, lo loguea en `activation_status_summary.dat` y continúa con la siguiente configuración en lugar de abortar el barrido.

11. Diagrama de flujo del solver

El pseudo-código de abajo sigue el solver real. El setup compartido es común a todos los métodos no lineales; luego se ejecuta uno de los tres bloques específicos según `nonlinearMethod`.

Algorithm 1 Setup compartido del paso de tiempo y commit/rollback final.

```

1: Leer diccionarios y ensamblar  $M, K, L = K/(\chi C_m)$ .
2: while  $t^n < T$  do
3:   Guardar snapshots:  $V^n, I_{\text{ion}}^n, s^n$ .
4:   Construir  $A = M/\Delta t - \theta L$  y  $B = M/\Delta t + (1 - \theta)L$ .
5:   Construir la fuente vieja  $S^n = -I_{\text{ion}}^n + I_{\text{ext}}^n/(\chi C_m)$ .
6:   if jfnkInitGuessOrder  $\geq 1$  y existe paso previo then
7:      $x_0 \leftarrow V^n + (\Delta t/\Delta t_{\text{prev}})(V^n - V^{n-1})$ .
8:     Recortar  $x_0$  a [jfnkInitGuessVmMin, jfnkInitGuessVmMax].
9:   else
10:     $x_0 \leftarrow V^n$ .
11:   end if
12:   Correr el Algoritmo 2, 3 o 4.
13:   if no convergió y nonlinearAcceptUnconverged es false then
14:     Hacer rollback de  $V_m, I_{\text{ion}}, s$  a los snapshots guardados en  $t^n$ .
15:   else
16:     Committear el iterado no lineal final.
17:   end if
18:   Actualizar tiempos de activación y muestras del benchmark.
19: end while

```

Algorithm 2 Solve no lineal de Picard.

Entrada: Candidato inicial $\mathbf{x}_0$, snapshot de state $\mathbf{s}^n$.

```
1: for  $k = 0, \dots, \text{nonlinearIterations} - 1$  do
2:   Guardar iterado previo  $\mathbf{x}_{k-1}$ ,  $I_{\text{ion},k-1}$ ,  $\mathbf{s}_{k-1}$  para los residuos.
3:   EVALUATENONLINEARFIELDS( $\mathbf{x}_k$ ): resetear states a  $\mathbf{s}^n$ , reconstruir  $V_m$  en los puntos de
   cuadratura de Iion si está habilitado, llamar a ionicModel->solveODE, computar  $I_{\text{ion},k}$ .
4:   Construir el RHS  $\mathbf{r}(I_{\text{ion},k})$  usando (30).
5:   Resolver  $\mathbf{A}\mathbf{V}_* = \mathbf{r}(I_{\text{ion},k})$  con linearSolver.
6:   Relajar:  $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \text{nonlinearRelaxation}(\mathbf{V}_* - \mathbf{x}_k)$ .
7:   Re-evaluar los ODEs y  $I_{\text{ion}}$  en  $\mathbf{x}_{k+1}$ .
8:   Computar  $R_{\text{cpl}}$ ,  $R_{V_m}$ ,  $R_{I_{\text{ion}}}$  y el opcional  $R_s$ .
9:   if todos los criterios pedidos pasan y  $k + 1 \geq \text{minNonlinearIterations}$  then
10:    Marcar convergido y salir del bucle.
11:   end if
12: end for
```

Algorithm 3 Solve no lineal diagonalIion.

Entrada: Candidato inicial $\mathbf{x}_0$, snapshot de state $\mathbf{s}^n$.

```
1: for  $k = 0, \dots, \text{nonlinearIterations} - 1$  do
2:   Evaluar states y  $I_{\text{ion},k}$  en  $\mathbf{x}_k$  exactamente como en Picard.
3:   Aproximar la derivada local  $S'_k \approx -[I_{\text{ion}}(\mathbf{x}_k + \varepsilon_d) - I_{\text{ion}}(\mathbf{x}_k)]/\varepsilon_d$ , con  $\varepsilon_d = \text{diagonalIionEpsilon}$ .
4:   Ensamblar  $\mathbf{A}_k = \mathbf{A} - \theta \text{diag}(S'_k)$ .
5:   Construir el RHS corregido  $\mathbf{r}_k = \mathbf{r}(I_{\text{ion},k}) - \theta \text{diag}(S'_k)\mathbf{x}_k$ .
6:   Resolver  $\mathbf{A}_k\mathbf{V}_* = \mathbf{r}_k$  con linearSolver.
7:   Relajar, re-evaluar ODEs/ $I_{\text{ion}}$ , computar residuos y testear convergencia como en Picard.
8: end for
```

Algorithm 4 Solve Jacobian-Free Newton–Krylov.

Entrada: Candidato inicial $\mathbf{x}_0$, snapshot de state $\mathbf{s}^n$.

```
1: for  $k = 0, \dots, \text{nonlinearIterations} - 1$  do
2:   Evaluar  $\mathbf{F}_k = \mathbf{F}(\mathbf{x}_k)$ ; esto resetea states a  $\mathbf{s}^n$ , resuelve los ODEs y recomputa  $I_{\text{ion}}$ .
3:   Computar residuos. Si todos los criterios pedidos pasan, marcar convergido y salir.
4:   Definir el producto matrix-free  $\mathcal{J}_k\mathbf{v} = [\mathbf{F}(\mathbf{x}_k + \varepsilon\mathbf{v}) - \mathbf{F}_k]/\varepsilon$ , con  $\varepsilon$  de jfnkEpsilon.
5:   Resolver  $\mathcal{J}_k\boldsymbol{\delta}_k = -\mathbf{F}_k$  por GMRES reiniciado usando jfnkMaxKrylovIterations, jfnkMaxRestarts
   y jfnkLinearTolerance.
6:   if jfnkLineSearch then
7:     Comenzar con  $\alpha = \text{nonlinearRelaxation}$  y retroceder hasta que Armijo (40) tenga éxito,
     se agote el presupuesto de intentos, o se alcance jfnkLineSearchAlphaMin.
8:   else
9:      $\alpha \leftarrow \text{nonlinearRelaxation}$ .
10:  end if
11:   $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \alpha\boldsymbol{\delta}_k$ .
12:  Re-evaluar  $\mathbf{F}(\mathbf{x}_{k+1})$ , actualizar campos, computar residuos y testear convergencia.
13: end for
```

Leyenda. Azul = constante Python fijada en `run.convergence.py`; verde = clave de diccionario OpenFOAM leída por el solver. Constantes y claves están mapeadas uno-a-uno en la sección 12.

12. Driver manufacturado: run_convergence.py

El script de convergencia `/home/pablo/OpenFOAM/pablo-v2312/run/tutorials_electro/highOrderManufacturedF` genera un caso MMS cuyo camino numérico busca coincidir con este solver. Las tablas de abajo describen las constantes expuestas ahí, las entradas de diccionario OpenFOAM que generan, y qué parte de las ecuaciones discretas afectan. Las mismas claves de diccionario las lee `highOrderElectroActivationFoamImplicitJfNK`; solo difieren ítems específicos del problema como campos exactos versus TNNP/estímulo.

12.1. Geometría y barrido de parámetros

Constante Python	Clave OpenFOAM	Ecuación / mecanismo afectado
DIMENSIONS	—	Selecciona el dominio manufacturado 2D o 3D y los stencils LRE correspondientes en <code>P_LEVELS_BY_DIM</code> .
N_VALUES	conteos de celdas de <code>blockMeshDict</code>	Refinamiento espacial. Cambia el error de discretización FV/LRE en el PDE, la cuadratura de Iion y la reconstrucción de states.
DT_VALUES	<code>controlDict.deltaT</code>	Paso de tiempo externo del PDE. También fija el intervalo de integración de cada solve de ODE local.
END_TIME	<code>controlDict.endTime</code>	Tiempo final del test MMS transitorio.
IMPLICIT_SCHEMES	<code>spatialIntegrationProperties</code>	Elige <code>implicitBackwardEuler</code> ($\theta = 1$) o <code>crankNicolson</code> ($\theta = 1/2$).
MASS_MATRIX_VALUES	<code>spatialIntegrationProperties</code>	Elige <code>Matrix</code> imped o consistente-LRE en $A = M/\Delta t - \theta L$.
ACTIVE_HO_PAIRS_Vm_States	<code>useHighOrder_Vm</code> <code>useHighOrder_Iion</code> <code>LRECoeffs_*</code>	/ Tupla (nivel-Vm, nivel-Iion/states). El nivel de Vm afecta la difusión y la masa consistente; el nivel de Iion/states afecta dónde se resuelven los ODEs y dónde se integra la fuente no lineal.
P_LEVELS_BY_DIM	<code>highOrderCoeffs</code>	Mahea <code>p1,p2,p3</code> al orden polinómico LRE <code>N</code> , objetivo de stencil <code>Nn</code> , exponente del peso radial <code>k</code> , y <code>maxStencilSize</code> .

12.2. Método no lineal y controles de convergencia

Constante Python	Clave OpenFOAM	Ecuación / mecanismo afectado
NONLINEAR_METHOD	<code>nonlinearMethod</code>	Selecciona la estrategia no lineal para el acoplamiento PDE/fuente: Picard, JFNK, o <code>diagonalIion</code> .

Constante Python	Clave OpenFOAM	Ecuación / mecanismo afectado
IMPLICIT_NONLINEAR_ITERATIONS	nonlinearIterations	Número máximo de correcciones no lineales por paso de tiempo del PDE.
IMPLICIT_MIN_NONLINEAR_ITERATIONS	minNonlinearIterations	Número mínimo de correcciones antes de que un test de convergencia pueda parar el bucle.
NONLINEAR_TOLERANCE	nonlinearTolerance	Tolerancia del residuo acoplado del PDE R_{cpl} . Usada directamente por JFNK y diagonal-ion; en el solver electro también se reporta para Picard.
NONLINEAR_VM_TOLERANCE	nonlinearVmTolerance	Corrección relativa de la incógnita del PDE V_m .
NONLINEAR_STATES_TOLERANCE	nonlinearStatesTolerance	Corrección relativa de todos los states del ODE. En el solver MMS estos son u_1, u_2, u_3 ; en este solver electro son las variables de state del TNNP.
NONLINEAR_REQUIRE_STATES_CONVERGENCE	nonlinearRequireStatesConvergence	Si es true, el residuo de states es parte del criterio de parada.
NONLINEAR_ACCEPT_UNCONVERGED	nonlinearAcceptUnconverged	Si es false, un solve no lineal electro fallido hace rollback de V_m, I_{ion} y states a t^n .
NONLINEAR_IION_TOLERANCE	nonlinearIionTolerance	Corrección relativa de la fuente no lineal I_{ion} .
NONLINEAR_RELAXATION	nonlinearRelaxation	Factor de mezcla Picard/diagonal o longitud de paso inicial de Newton para JFNK/Armi-jo.

12.3. Controles de JFNK y de diagonal-ion

Constante Python	Clave OpenFOAM	Ecuación / mecanismo afectado
JFNK_MAX_KRYLOV_ITERATIONS	jfnkMaxKrylovIterations	Longitud de reinicio m de GMRES(m), controlando la memoria y la calidad de la aproximación de Krylov.
JFNK_MAX_RESTARTS	jfnkMaxRestarts	Número de veces que la base de GMRES puede descartarse y reconstruirse.
JFNK_LINEAR_TOLERANCE	jfnkLinearTolerance	Tolerancia relativa del solve de la corrección de Newton $J\delta = -F$.

Constante Python	Clave OpenFOAM	Ecuación / mecanismo afectado
JFNK_EPSILON	jfnkEpsilon	Perturbación base para los productos matriz-vector matrix-free del Jacobiano. Debe estar por encima del piso de ODE/-ruido pero suficientemente chica para aproximar una derivada.
JFNK_INIT_GUESS_ORDER	jfnkInitGuessOrder	Elige el V_m inicial retrasado o linealmente extrapolado para el solve no lineal.
JFNK_INIT_GUESS_VM_MIN/MAX	jfnkInitGuessVmMin/Max	Intervalo de clamp para el valor inicial extrapolado y, si está habilitado, las entradas del ODE.
JFNK_CLAMP_ODE_INPUT	jfnkClampODEInput	Protege el solve del ODE de sondas de diferencias finitas de JFNK no físicas.
JFNK_LINE_SEARCH	jfnkLineSearch	Activa/desactiva el retroceso de Armijo.
JFNK_LINE_SEARCH_MAX_ITER	jfnkLineSearchMaxIter	Número máximo de divisiones por la mitad del paso de Newton.
JFNK_LINE_SEARCH_ALPHA_MIN	jfnkLineSearchAlphaMin	Piso para la longitud de paso de Armijo.
JFNK_ARMIJO_C	jfnkArmijoC	Parámetro de descenso suficiente en (40).
DIAGONAL_IION_EPSILON	diagonalIionEpsilon	Paso de diferencias finitas para la derivada diagonal de la fuente en diagonalIion.

12.4. Solver lineal del PDE

Constante Python	Clave OpenFOAM	Ecuación / mecanismo afectado
IMPLICIT_LINEAR_SOLVER	linearSolver	Solver disperso Eigen usado por Picard y diagonalIion para $AV = r$. JFNK en cambio usa GMRES sobre la envoltura Jacobian-free.
IMPLICIT_TOLERANCE	tolerance	Tolerancia de BiCGSTAB para el solve lineal del PDE.
IMPLICIT_MAX_ITERATIONS	maxIterations	Tope de iteraciones de BiCGSTAB.

12.5. Integrador del ODE de states

Constante Python	Clave OpenFOAM	Ecuación / mecanismo afectado
STATE_ODE_SOLVER	stateODESolver timeIntegrationProperties.solver	/ Integrador de ODE local para states MMS o states TNNP.

Constante Python	Clave OpenFOAM	Ecuación / mecanismo afectado
STATE_ODE_ABS_TOL	stateODEAbsTol timeIntegrationProperties.absTol	/ Tolerancia absoluta del solve del ODE.
STATE_ODE_REL_TOL	stateODERelTol timeIntegrationProperties.relTol	/ Tolerancia relativa del ODE. Controla el piso de ruido visto por las diferencias finitas de JFNK.
STATE_ODE_INITIAL_STEP	stateODEInitialStep initialODEStep	/ Primer sub-paso RKF45 intentado dentro de un paso de tiempo del PDE.
STATE_ODE_MAX_STEPS	stateODEMaxSteps / maxSteps	Máximo de sub-pasos de ODE por solve local.

12.6. Parámetros LRE de alto orden por nivel-p

P_LEVELS_BY_DIM[dim][p] da, para cada (dim, p): N (orden polinómico), Nn (vecinos del stencil), k (radio de búsqueda), maxStencilSize (tope). El script escribe estos en un bloque anidado highOrderCoeffs{LRECoeffs_Vm{...}; LRECoeffs_Iion{...}} cuando HO está habilitado para Vm o I_{ion} .

12.7. Diagnósticos y control de corrida

Constante Python	Clave OpenFOAM	Mecanismo afectado
WRITE_NONLINEAR_RESIDUALS	writeNonlinearResiduals	Log de residuos on/off.
KEEP_LOGS_ON_SUCCESS	—	Switch solo-Python: conservar logs del solver/blockMesh tras corridas exitosas. Los logs de corridas fallidas se conservan siempre.

Para el tutorial de electro-activación el driver análogo también expone parámetros de umbral de activación, parada en P8 y muestreo de la diagonal. No afectan el solve PDE/ODE en sí; afectan solo la parada y el post-procesamiento de los tiempos de activación.

13. Resultados

Esta sección reúne los resultados de convergencia actuales para los benchmarks 2D y 3D estilo Niederer, corridos con las tres estrategias no lineales expuestas por el solver (Picard, diagonalIion, JFNK). Todas las corridas usan el esquema temporal Crank–Nicolson ($\theta = 1/2$), la matriz de masa consistente de alto orden cuando la LRE está habilitada, el integrador de ODE de states RKF45, y un único $\Delta t_{req} = 10^{-4}$ s solicitado. Se muestrean seis pares LRE (V_m, I_{ion}): N0/N0, N0/p1, N0/p3, p3/N0, p3/p1, p3/p3. Cada tabla lista, para la métrica de su caption, el valor de cada corrida disponible; NC marca las combinaciones que aún no se han corrido. Los resultados 2D viven bajo highOrderNiedererEtAl2012ImplicitJfNK[_diagonalIion|_Picard]/results/example0/2D/; los resultados 3D bajo los mismos árboles pero con el sufijo 3D/. El tiempo de reloj de pared (wall-clock) es el tiempo transcurrido reportado por /usr/bin/time -v, redondeado a minutos enteros. El RSS pico es el tamaño máximo del conjunto residente (resident set size) del proceso del solver, expresado en MiB ($\lfloor \text{kB}/1024 \rfloor$).

13.1. Benchmark 2D estilo Niederer

El caso 2D sigue la reducción planar de 20×8 mm usada por los reportes explícito e implícito no-JfNK (una sola celda en z , fibras planares), ejercitado a tamaños de malla $\Delta x \in \{0,5, 0,25, 0,125, 0,1\}$ mm; solo el barrido Picard incluye actualmente la malla más fina.

Cuadro 7: Tiempo de activación P8 2D [ms] a $\Delta t = 10^{-4}$ s. Las columnas son tamaños de malla Δx en mm.

Vm	Iion	0,5	0,25	0,125	0,1
<i>Picard</i>					
NO	NO	153.32	62.99	45.03	42.76
NO	p1	91.05	54.86	43.21	41.84
NO	p3	32.46	39.14	39.83	NC
p3	NO	180.55	68.84	48.83	NC
p3	p1	68.92	58.81	46.33	NC
p3	p3	33.09	40.22	40.84	NC
<i>diagonalIion</i>					
NO	NO	153.16	62.86	44.92	NC
NO	p1	90.92	54.74	43.08	NC
NO	p3	32.37	38.97	39.66	NC
p3	NO	180.42	68.71	48.67	NC
p3	p1	68.83	58.70	46.20	NC
p3	p3	33.00	40.03	40.65	NC
<i>JfNK</i>					
NO	NO	153.13	62.85	44.87	NC
NO	p1	90.92	54.72	NC	NC
NO	p3	32.36	38.92	39.59	NC
p3	NO	180.38	68.69	NC	NC
p3	p1	68.83	NC	NC	NC
p3	p3	33.00	NC	NC	NC

Cuadro 8: Tiempo de reloj de pared del solver 2D [min] a $\Delta t = 10^{-4}$ s (redondeado a minutos enteros).

Vm	Iion	0,5	0,25	0,125	0,1
<i>Picard</i>					
NO	NO	15	31	83	125
NO	p1	20	51	141	197
NO	p3	23	102	402	NC
p3	NO	18	40	112	NC
p3	p1	20	60	176	NC
p3	p3	22	105	419	NC
<i>diagonalIion</i>					
NO	NO	18	31	81	NC
NO	p1	22	52	147	NC
NO	p3	29	131	533	NC
p3	NO	21	38	107	NC
p3	p1	23	59	164	NC
p3	p3	28	133	549	NC
<i>JfNK</i>					
NO	NO	59	155	544	NC
NO	p1	82	263	NC	NC
NO	p3	124	647	3407	NC
p3	NO	67	169	NC	NC
p3	p1	87	NC	NC	NC
p3	p3	131	NC	NC	NC

Cuadro 9: Tamaño de conjunto residente pico 2D [MiB] a $\Delta t = 10^{-4}$ s.

Vm	lion	0,5	0,25	0,125	0,1
<i>Picard</i>					
NO	NO	148	152	174	194
NO	p1	149	157	191	219
NO	p3	155	177	288	NC
p3	NO	155	191	346	NC
p3	p1	156	194	358	NC
p3	p3	161	218	457	NC
<i>diagonalIion</i>					
NO	NO	148	153	176	NC
NO	p1	149	158	194	NC
NO	p3	155	181	296	NC
p3	NO	155	191	347	NC
p3	p1	156	195	365	NC
p3	p3	161	218	457	NC
<i>JFNK</i>					
NO	NO	148	153	173	NC
NO	p1	149	158	NC	NC
NO	p3	155	181	299	NC
p3	NO	152	187	NC	NC
p3	p1	153	NC	NC	NC
p3	p3	160	NC	NC	NC

Discusión de los resultados 2D. A $(\Delta x, \Delta t, \text{par LRE})$ fijos las tres estrategias no lineales dan tiempos de activación que coinciden dentro de $\sim 0,1\text{--}0,2$ ms; p. ej. a $\Delta x = 0,125$ mm con (NO, NO) los valores son 45,03/44,92/44,87 ms para Picard, diagonalIion, JFNK respectivamente. La elección del acoplamiento no lineal 2D es por lo tanto esencialmente *gratis* desde el punto de vista de la precisión aquí — los errores relevantes son espaciales y temporales, no errores de iteración no lineal. Los tres métodos difieren marcadamente en costo: JFNK es $\sim 3\text{--}6\times$ más caro que Picard para el mismo caso a $\Delta x = 0,5$ mm y crece más rápido que linealmente con el tamaño de malla, porque cada producto matriz–vector de Krylov realiza una re-evaluación completa del ODE ($\Delta x = 0,125$, (NO, p3): 402 min para Picard, 533 min para diagonalIion, 3407 min — $\sim 2,4$ días — para JFNK). El RSS pico está dominado por el stencil LRE y es esencialmente independiente del método; la columna JFNK es levemente menor para algunas entradas porque la envoltura matrix-free evita la corrección diagonal y el workspace explícito de BiCGSTAB.

13.2. Benchmark 3D estilo Niederer

El barrido 3D sigue la geometría de losa publicada de Niederer ($20 \times 3 \times 7$ mm, $40 \times 6 \times 14$ celdas a $\Delta x = 0,5$ mm). Al momento de escribir, solo los barridos diagonalIion y Picard incluyen corridas 3D, todas a $\Delta x = 0,5$ mm. El barrido JFNK 3D aún no se ha corrido.

Cuadro 10: Tiempo de activación P8 3D [ms] a $\Delta x = 0,5$ mm, $\Delta t = 10^{-4}$ s.
Las columnas son los métodos no lineales disponibles.

Vm	lion	Picard	diagonalIion
NO	NO	136.67	136.61
NO	p1	33.50	33.39
NO	p3	30.59	30.51
p3	NO	149.13	149.06
p3	p1	32.44	32.36
p3	p3	29.63	NC

Cuadro 11: Tiempo de reloj de pared del solver 3D [min] a $\Delta x = 0,5$ mm, $\Delta t = 10^{-4}$ s.

Vm	Iion	Picard	diagonalIion
NO	NO	80	82
NO	p1	168	229
NO	p3	1489	2159
p3	NO	163	143
p3	p1	167	269
p3	p3	1487	NC

Cuadro 12: Tamaño de conjunto residente pico 3D [MiB] a $\Delta x = 0,5$ mm, $\Delta t = 10^{-4}$ s.

Vm	Iion	Picard	diagonalIion
NO	NO	156	157
NO	p1	211	211
NO	p3	787	849
p3	NO	1123	1121
p3	p1	1164	1162
p3	p3	1565	NC

Discusión de los resultados 3D. En la losa 3D gruesa ($\Delta x = 0,5$ mm), activar la LRE solo en la fuente iónica cambia el tiempo de activación de $\approx 136,7$ ms (NO/NO) a $\approx 33,5$ ms (NO/p1) y $\approx 30,6$ ms (NO/p3) — el mismo efecto cualitativo de alto-orden-en-la-fuente ya visto en el barrido 2D y en los reportes Niederer explícito/implícito. El caso totalmente de alto orden (p3/p3) alcanza 29,63 ms con Picard (la contraparte diagonalIion aún no se ha completado). Los dos métodos no lineales nuevamente coinciden mejor que 0,1 ms para cada entrada donde ambos están disponibles; Picard es generalmente más rápido, con la brecha más notable en el caso más pesado (NO/p3: 1489 min para Picard vs. 2159 min para diagonalIion). El RSS pico escala con el tamaño del stencil LRE y es esencialmente independiente del método. El conjunto de datos es demasiado ralo en Δx para sacar conclusiones cuantitativas sobre convergencia de malla en 3D; la brecha inmediata a llenar es la entrada faltante p3/p3 de diagonalIion y un barrido JFNK 3D a la misma malla.

A. Tabla de dimensiones

Campo	dimensionSet de OpenFOAM	SI
Vm	[1, 2, -3, 0, 0, -1, 0]	volt
Iion	dimVoltage/dimTime	$V s^{-1}$
externalStimulusCurrent	dimCurrent/dimVolume	$A m^{-3}$
conductivity	dimCurrent^2 dimTime^3 / (dimMass dimVolume)	$S m^{-1}$
chi	dimless/dimLength	m^{-1}
Cm	dimCurrent dimTime / (dimVoltage dimArea)	$F m^{-2}$

B. Mapa de archivos

Ruta (relativa a cardiacFoam-pc/)	Rol
<code>applications/solvers/.../JfNK.C</code>	Unidad de traducción principal, incluyendo el kernel CellML TNNP embebido, la actualización de states, la evaluación de corriente, y la integración local del ODE.
<code>applications/solvers/.../createFields.H</code>	Inicialización de campos y lectura de diccionarios.
<code>tutorials_electro/.../run_convergence.py</code>	Script driver.
<code>constant/cardiacProperties</code>	χ , C_m , $\mathbf{D}$, modelo iónico.
<code>constant/timeIntegrationProperties</code>	Solver de ODE, tolerancias, tope de subpasos.
<code>constant/spatialIntegrationProperties</code>	Esquema temporal, flags HO, opciones de JFNK / lineal / búsqueda de línea.
<code>constant/stimulusProtocol</code>	Regiones y temporización del estímulo.

C. Diferencias vs. el hermano PETSc

Algorítmicamente los dos solvers son idénticos. Las únicas diferencias están en el back-end de álgebra lineal y unas pocas perillas visibles al usuario:

Aspecto	Este solver (Eigen)	Hermano PETSc
Contenedor de matriz dispersa	<code>Eigen::SparseMatrix</code>	Mat de PETSc
Solver directo disperso	<code>Eigen::SparseLU</code>	KSPPREONLY+PCLU
Solver iterativo (rama Picard)	<code>Eigen::BiCGSTAB+ILUT</code>	KSP+PC (catálogo completo de PETSc)
Krylov interno de JFNK	GMRES propio	KSPGMRES sobre MatShell
Precondicionador dentro de JFNK	ninguno (solo Krylov)	cualquier PC de PETSc
Dependencia en tiempo de compilación	Eigen3 header-only	PETSc + MPI
Soporte de corrida paralela	solo serial	MPI-paralelo de fábrica

Todos los parámetros relacionados a la convergencia (`nonlinearTolerance`, `jfnkEpsilon`, `jfnkLineSearch*`, `jfnkInitGuess*`, `jfnkClampODEInput`, `nonlinearAcceptUnconverged`, `ODE*`) tienen los mismos nombres y significados en ambos solvers, así que la misma configuración de `run_convergence.py` produce una corrida algorítmicamente equivalente con cualquiera de los dos back-ends (salvo las diferencias de rendimiento y de escalado paralelo).

Referencias

- [1] K. H. W. J. ten Tusscher, D. Noble, P. J. Noble, A. V. Panfilov, *A model for human ventricular tissue*, AJP – Heart and Circulatory Physiology **286** (2004) H1573–H1589.
- [2] S. A. Niederer et al., *Verification of cardiac tissue electrophysiology simulators using an N-version benchmark*, Prog. Biophys. Mol. Biol. **104** (2012) 22–48.
- [3] D. A. Knoll, D. E. Keyes, *Jacobian-Free Newton-Krylov methods: a survey of approaches and applications*, J. Comp. Phys. **193** (2004) 357–397.
- [4] J. Nocedal, S. J. Wright, *Numerical Optimization*, 2nd edition, Springer, 2006.
- [5] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd edition, SIAM, 2003.
- [6] E. Hairer, G. Wanner, *Solving Ordinary Differential Equations II*, Springer, 1996.